

第八章：数据库编程——嵌入式 SQL

根本矛盾

SQL 和 C/Java 是两种完全不同的语言：

- SQL 是集合语言——一句话能处理一堆数据 (SELECT * FROM Student)
- C/Java 是过程语言——一次只能处理一条数据

这两种思维天然矛盾。怎么让它们在同一个程序里协作？

关键认知：C 程序是主控者（项目经理），SQL 是被调用的工具（数据库专家）。C 程序 → 调用 → SQL → 操作 → 数据库。

而"协作"靠 3 个通信机制：SQL 通信区、主变量、游标。第八章就是讲这 3 件事 + 嵌入式 SQL 语法 + 完整程序长什么样。

一、嵌入式 SQL 语约定

1.1 什么是"嵌入式 SQL"

嵌入式 SQL = 把 SQL "嵌入/塞进" C/Java 程序里。SQL 语句前面加一个特殊标记，预编译器看到标记就知道"这是 SQL，不是 C/Java"。

1.2 两种语言的语法

主语言	嵌入式 SQL 格式	例子
C 语言	<code>`EXEC SQL <SQL 语句>;`</code>	<code>`EXEC SQL SELECT * FROM Student WHERE Sno = :sno;`</code>
Java	<code>`#SQL{<SQL 语句>;}`</code>	<code>`#SQL{SELECT * FROM Student WHERE Sno = :sno};`</code>

关键点：EXEC SQL / #SQL{...} 这些标记是给预编译器看的，不是给 C 编译器看的。预编译器先把这些行翻译成 C/Java 的 DBMS 调用函数，再交给 C/Java 编译器。

1.3 几个易混淆的"SQL 概念"（本章不深入，区分清楚即可）

名称	含义	本章涉及
嵌入式 SQL	在 C/Java 里调用 SQL	是（本章主角）
交互式 SQL	直接在 DBMS 命令行里敲 SQL	否
静态 SQL	编译期就确定的 SQL（写死的）	嵌入式 SQL 通常是静态的
动态 SQL	运行时拼接的 SQL（字符串拼接）	源材料未深入
过程化 SQL	SQL 本身支持过程逻辑（如 PL/SQL 的 BEGIN...END 块）	源材料未提，本章不展开

二、SQL 通信区 (SQLCA) —— SQL 向主语言"汇报"状态

2.1 什么是 SQLCA

源材料说："用 EXEC SQL INCLUDE SQLCA 加以定义"。

- INCLUDE = 预编译指令，相当于 C 语言的 #include，把 SQLCA 的定义复制到当前位置
- SQLCA = DBMS 预定义的结构体（struct），包含 SQLCODE、SQLERRM、SQLWARN 等字段

2.2 SQLCODE 三态（高频考点）

SQLCODE 值	含义	例子
= 0	SQL 执行成功	SELECT 找到数据
> 0	SQL 执行成功但有警告	100 = 找不到更多数据（FETCH 完时）
< 0	SQL 执行失败	违反约束、连接断开等

程序实例第 237 行：

```
if (SQLCA.SQLCODE != 0) break;
```

- SQLCA.SQLCODE 是 C 语言结构体成员访问（不是嵌入式 SQL 语法）
- 含义："如果 SQLCODE 不等于 0，跳出循环"——即"没数据了或出错了"

2.3 为什么要 SQLCA

C/Java 有"异常处理"机制（try-catch），但 SQL 不会"抛异常"——它只是把状态码塞进 SQLCA 让 C 程序自己查。这是"集合语言 vs 过程语言"通信的第一个工具。

三、主变量与指示变量 —— 双向传值的桥梁

3.1 主变量

主变量 = SQL 语句里使用的主语言程序变量（即 SQL 借用了 C/Java 的变量）。

方向	含义	例子
输入主变量	由应用程序赋值，SQL 引用	`SELECT * FROM Student WHERE Sno = :sno;` 里的 `:sno`
输出主变量	SQL 赋值或设状态，返回应用程序	`FETCH SX INTO :HSno, :HSname, :HSage;` 里的 `:HSno` 等

3.2 主变量的两个约定

约定 1：主变量名前面要加冒号：（作为 SQL 与主语言变量的标志，避免歧义）。

约定 2：所有主变量必须在 `BEGIN DECLARE SECTION` 与 `END DECLARE SECTION` 之间声明：

```
EXEC SQL BEGIN DECLARE SECTION;
char Deptname[20];
char HSno[9];
int HSage;
EXEC SQL END DECLARE SECTION;
```

这是预编译器的要求——预编译器要扫这一段，把主变量翻译成 DBMS 调用参数。

3.3 指示变量 —— 处理 NULL 的关键

源材料第 183 行说指示变量"用来指示所指主变量的值或条件"——这个说法很模糊。真实含义是三态：

指示变量值	含义
-------	----



< 0	主变量值为 NULL
= 0	主变量值正常
> 0	主变量值被截断（字符串超长）

典型场景：

```
SELECT Sno, Grade FROM SC WHERE ...;
```

如果 Grade 列是 NULL，主变量无法区分"NULL"和"0"——这时就用指示变量区分。

指示变量必须：

- 是整型（int）
- 与主变量一起在 DECLARE SECTION 中声明

四、游标（Cursor）—— 集合 vs 记录的核心协调机制

4.1 为什么需要游标

SQL 是面向集合的——一条 SELECT 可能返回 1000 条记录。

C 是面向记录的——一组变量一次只能放 1 条记录。

怎么把 1000 条记录逐条交给 C 处理？——引入游标。

4.2 游标是什么

游标 = SQL 系统为嵌入式程序提供的"结果集访问机制"，本质是一个数据缓冲区，用来存放 SQL 语句的执行结果，每个游标区有自己的名字。

- 不是 C 程序的变量
- 不是 内存地址（C 指针）
- 是 SQL 系统专门维护的一个逻辑标记，指向结果集的"当前行"

4.3 游标的 4 步工作流（核心考点）

步骤	干什么	类比（书页/书签）
DECLARE 游标	声明游标和对应 SQL，不执行	告诉书架"我要读《数据库系统概论》"
OPEN 游标	真正执行 SQL，游标指向第一行之前	打开书到第一页
FETCH 游标	取当前行到主变量，游标推进一行	翻一页
CLOSE 游标	释放游标	合上书

循环 FETCH 就是逐行处理结果集：

```
EXEC SQL OPEN SX;    // 打开游标
for (;;) {           // 无限循环
    EXEC SQL FETCH SX INTO :HSno, :HSname, :HSsex, :HSage;
    if (SQLCA.SQLCODE != 0) break; // SQLCODE != 0 退出（100 表示取完了）
    /* 处理这一行 ... */
}
EXEC SQL CLOSE SX;   // 关闭游标
```

4.4 游标的高级用法：WHERE CURRENT OF

```
EXEC SQL UPDATE Student SET Sage = :NEWAGE WHERE CURRENT OF SX;
```

WHERE CURRENT OF 游标名 = "更新当前游标指向的那一行"，不是按某个条件匹配，而是按位置。



应用场景：在 FETCH 循环里，用户确认要修改当前学生年龄时，用 WHERE CURRENT OF 直接更新这一行。

4.5 游标类比（不用 C 指针）

比喻	含义
书页 + 书签	游标 = 书签当前停在第几页，FETCH = 翻一页
队列的队首指针	游标指向"当前要处理的行"
绝对不要用 C 指针	C 指针是内存地址，可任意 +/-；游标是 SQL 的逻辑标记，只能 FETCH 一次推一行

恍然大悟点：游标不存在 C 语言里，只存在于 SQL 系统中。C 程序看到的只是一个"名字"（SX），背后由 SQL 引擎维护"当前行"的位置。

五、连接与关闭 —— 嵌入式 SQL 访问数据库的"门"

5.1 必须先连接

嵌入式 SQL 访问数据库必须先连接。

```
EXEC SQL CONNECT TO target [AS connection-name] [USER user-name];
```

- target：要连接的数据库服务器（如 TEST@localhost:54321）
- connection-name：可选的连接名（给连接起个名字）
- user-name：登录用户名/密码

5.2 切换连接（SET CONNECTION）

如果程序要连多个数据库，可在运行时切换当前连接：

```
EXEC SQL SET CONNECTION connection-name DEFAULT;
```

5.3 关闭连接

```
EXEC SQL DISCONNECT [connection];
```

connection 可省略，默认关闭当前连接。

六、程序实例分步解读

源材料给了一个完整 C 程序，但 OCR 中有多处错误。下面先列 OCR 错误修正对照表，再给出修正版和分步解读。

6.1 OCR 错误修正对照表

以下表格左侧"源材料原文"列照录了 OCR 中的标点错误（含全角符号、中英文混用等），供对照参考。

源材料原文（错误）	应该是	错误类型
`USER "SYSTEM"/"MANAGER", `	`USER "SYSTEM"/"MANAGER"``	标点错（逗号→无）



`FETCH SX INTO :HSno, : Hsname, HSsex, ; HSage`	`FETCH SX INTO :HSno, :HSname, :HSsex, :HSage`	中英文标点混用、缺冒号
`SQLCA.SQLCODE! =0`	`SQLCA.SQLCODE != 0`	全角 `! =` → 半角 `!=`
`"n%-10s ... In"`	`"\n%-10s ... \n"`	缺反斜杠 `n` → `\\n` (换行)
`Fssex`	`HSsex`	变量名 OCR 错
`yn! =Y`	`yn != 'Y`	缺左引号、全角叹号
`yn-=yllyn Y`	`yn == 'y`	
`"%os"、`"%od"``	`"%s"、`"%d"``	`0` → 无 (OCR 误加)
` (void) ` ` {} ` ` ( ; ; ) `	`(void)` `{} ` `(:;)`	全角括号 → 半角

6.2 修正版完整程序



```

EXEC SQL BEGIN DECLARE SECTION;
/* 主变量说明开始 */
char Deptname[20];
char HSno[9];
char HSname[20];
char HSsex[2];
int HSage;
int NEWAGE;
EXEC SQL END DECLARE SECTION;
/* 主变量说明结束 */

long SQLCODE;
EXEC SQL INCLUDE SQLCA;
/* 定义 SQL 通信区 */

int main(void) { /* C 语言主程序开始 */
    int count = 0;
    char yn; /* 变量 yn 代表 yes 或 no */

    printf("Please choose the department name (CS/MA/IS): ");
    scanf("%s", Deptname); /* 为主变量 Deptname 赋值 */

    EXEC SQL CONNECT TO TEST@localhost:54321 USER "SYSTEM"/"MANAGER";
    /* 连接数据库 TEST */

    EXEC SQL DECLARE SX CURSOR FOR
    /* 定义游标 SX */
        SELECT Sno, Sname, Ssex, Sage
        /* SX 对应的语句 */
        FROM Student
        WHERE SDept = :Deptname;

    EXEC SQL OPEN SX;
    /* 打开游标 SX，指向查询结果的第一行 */

    for (;;) { /* 用循环结构逐条处理结果集中的记录 */
        EXEC SQL FETCH SX INTO :HSno, :HSname, :HSsex, :HSage;
        /* 推进游标，将当前数据放入主变量 */

        if (SQLCA.SQLCODE != 0)
            /* SQLCODE != 0，表示操作不成功（100 = 无更多数据） */
            break;
        /* 利用 SQLCA 中的状态信息决定何时退出循环 */

        if (count++ == 0)
            /* 如果是第一行的话，先打出行头 */
            printf("\n%-10s %-20s %-10s %-10s\n",
                "Sno", "Sname", "Ssex", "Sage");

        printf("%-10s %-20s %-10s %-10d\n",
            HSno, HSname, HSsex, HSage);
        /* 打印查询结果 */

        printf("UPDATE AGE (y/n)? ");
        /* 询问用户是否要更新该学生的年龄 */
        do { scanf("%c", &yn); }
        while (yn != 'N' && yn != 'n' && yn != 'Y' && yn != 'y');

        if (yn == 'y' || yn == 'Y') { /* 如果选择更新操作 */
            printf("INPUT NEW AGE: ");
            scanf("%d", &NEWAGE);
            /* 用户输入新年龄到主变量中 */

            EXEC SQL UPDATE Student
            /* 嵌入式 SQL 更新语句 */
                SET Sage = :NEWAGE
                WHERE CURRENT OF SX;
            /* 对当前游标指向的学生年龄进行更新 */
        }
    }
}

```



```

    }
}

EXEC SQL CLOSE SX;
/* 关闭游标 SX，不再和查询结果对应 */

EXEC SQL COMMIT WORK;
/* 提交更新（WORK 关键字可省略） */

EXEC SQL DISCONNECT TEST;
/* 断开数据库连接 */
}

```

6.3 完整程序的 8 步生命周期

步骤	语句	干什么
① 声明主变量	<code>`BEGIN DECLARE SECTION ... END DECLARE SECTION`</code>	划定主变量范围
② 定义 SQLCA	<code>`EXEC SQL INCLUDE SQLCA`</code>	引入状态结构体
③ 连接数据库	<code>`EXEC SQL CONNECT TO ...`</code>	登录数据库
④ 声明游标	<code>`EXEC SQL DECLARE SX CURSOR FOR ...`</code>	声明 + 关联 SQL（不执行）
⑤ 打开游标	<code>`EXEC SQL OPEN SX`</code>	执行 SQL，游标指向第一行之前
⑥ 循环 FETCH	<code>`EXEC SQL FETCH SX INTO ...`</code>	逐行取数据 + 通过 SQLCODE 判退出
⑦ 关闭游标	<code>`EXEC SQL CLOSE SX`</code>	释放游标
⑧ 提交 + 断开	<code>`COMMIT WORK` / `DISCONNECT`</code>	提交事务 + 断开连接

COMMIT WORK 中 WORK 关键字可省略——标准 SQL 中 COMMIT 和 COMMIT WORK 等价。

七、跨章关联

第八章是"应用层"章节，和后续章节的连接：

关联章节	关联点
第 4 章 安全性	嵌入式 SQL 用 <code>`USER user-name`</code> 登录，本质是身份鉴别（第 4 章）
第 5 章 完整性	嵌入式 SQL 的 UPDATE 受完整性约束检查（主码、参照、用户定义完整性）
第 10 章 恢复	<code>`COMMIT/ROLLBACK`</code> 是事务的提交/回滚——参见第 10 章
第 11 章 并发	嵌入式 SQL 一个 <code>`main`</code> 函数通常是一个事务——并发控制见第 11 章

易混淆对比总结

表 1：3 个通信机制

通信机制	通信方向	解决问题	类比
------	------	------	----



SQL 通信区 (SQLCA)	SQL → 主语言 (状态汇报)	"SQL 执行得怎么样"	员工的工作汇报单
主变量	主语言 ↔ SQL (双向传值)	"传什么参数、取什么结果"	工作的具体内容 (任务/成果)
游标	SQL → 主语言 (结果集)	"一堆数据怎么一条条交"	翻书页的书签

表 2：输入主变量 vs 输出主变量

维度	输入主变量	输出主变量
赋值方	应用程序	SQL
引用方	SQL	应用程序
例子	`WHERE Sno = :sno` 里的 `:sno`	`FETCH ... INTO :HSno` 里的 `:HSno`
比喻	"给员工的输入材料"	"员工产出的工作成果"

表 3：嵌入式 SQL vs 几个"近亲" SQL 概念

概念	含义	与嵌入式 SQL 关系
嵌入式 SQL	在 C/Java 里调用 SQL	本章主角
交互式 SQL	在 DBMS 命令行里直接敲	不是嵌入式，单独运行
静态 SQL	编译期就确定的 SQL (写死)	嵌入式 SQL 通常是静态的
动态 SQL	运行时拼接的 SQL	嵌入式 SQL 也可动态
过程化 SQL (PL/SQL, T-SQL)	SQL 本身支持过程逻辑 (BEGIN...END 块)	完全不同的范式——SQL 内嵌过程，不是 SQL 被 C 嵌入

表 4：指示变量三态

指示变量值	含义	典型场景
< 0	主变量值为 NULL	选课表里 Grade = NULL (学生没成绩)
= 0	主变量值正常	正常 SELECT 出非空值
> 0	主变量值被截断	字符串超长，截断到主变量能存的长度

表 5：游标 4 步工作流 vs 程序 4 步生命周期

维度	游标 4 步	程序 4 步生命周期
范围	游标本身	嵌入式 SQL 程序整体
步骤	DECLARE → OPEN → FETCH → CLOSE	CONNECT → DECLARE/OPEN/FETCH/CLOSE → COMMIT → DISCONNECT
关系	包含在程序的"4 步"里	是游标 + 连接 + 事务的组合

逻辑主线



SQL (集合语言) vs C/Java (过程语言) —— 根本矛盾



解决思路: C 程序是主控者, SQL 是被调用的工具, 靠 3 个通信机制协作



嵌入式 SQL 语法 (C: EXEC SQL; Java: #SQL) —— 给预编译器看的



3 个通信机制

- |— SQL 通信区 (SQLCA) —— SQL 汇报状态
 - | — SQLCODE 三态: =0 成功 / >0 警告 (100=无更多数据) / <0 失败
- |— 主变量 —— 双向传值 (输入/输出)
 - | |— 冒号: 标志
 - | |— BEGIN/END DECLARE SECTION 中声明
 - | |— 指示变量: 整型, 三态 (NULL/正常/截断)
- |— 游标 —— 集合 vs 记录的核心协调
 - | — 4 步 workflow: DECLARE → OPEN → FETCH → CLOSE
 - | — 位置感: 游标指向当前行
 - | — 高级用法: WHERE CURRENT OF 游标名 (当前位置更新)



连接管理

- |— CONNECT TO target —— 登录
- |— SET CONNECTION —— 切换
- |— DISCONNECT —— 断开



完整程序生命周期 (8 步)

- ① 声明主变量
- ② 定义 SQLCA
- ③ 连接数据库
- ④ 声明游标
- ⑤ 打开游标
- ⑥ 循环 FETCH (用 SQLCODE 判退出)
- ⑦ 关闭游标
- ⑧ COMMIT + DISCONNECT



跨章呼应: 身份鉴别 (第 4 章) / 完整性检查 (第 5 章) / 事务 (第 10 章) / 并发 (第 11 章)

整个第八章就一件事: 让 C/Java 这种"过程语言"通过 3 个通信机制 (SQLCA / 主变量 / 游标) 调用"集合语言 SQL", 完成数据库编程。

